- *Fault*—A fault is a defect that produces an incorrect internal state either in the software or the infrastructure. For example, a method call returns a null value, even if its specification mandates that a non-null value is returned.
- *Error*—An error is a discrepancy between the expected behavior of a system and the actual one. For example, due to the preceding fault, a `NullPointerException` is thrown.
- *Failure*—When a fault is triggered and results in an error, a failure might occur, making the system unresponsive and unable to behave according to its specifications. For example, if the `NullPointerException` is not caught, the error provokes a failure: the system responds to any request with a 500 response.

Faults can become errors, which may provoke failures, so we should design applications to be *fault tolerant*. An essential part of resilience is ensuring that a failure will not cascade to other components of the system but stay isolated while it gets fixed. We also want the system to be *self-repairing* or *self-healing*, and the cloud model can enable that.

Throughout the book, I'll show you some techniques for tolerating faults and preventing their effects from propagating to other parts of the system and spreading the failure. For example, we'll use patterns like circuit breakers, retries, timeouts, and rate limiters.

1.3.4 Observability

Observability is a property that comes from the world of control theory. If you consider a system, observability is a measure of how well you can infer its internal state from its external outputs. In the software engineering context, the system is a single application or a distributed system as a whole.